in the global list as before and added to the local list of the next thread doing a reclamation process.

Another downside of hazard pointers is that they're covered by a patent application submitted by IBM.[2] If you write software for use in a country where the patents are valid, you need to make sure you have a suitable licensing arrangement in place. This is something common to many of the lock-free memory reclamation techniques; this is an active research area, so large companies are taking out patents where they can. You may well be asking why I've devoted so many pages to a technique that many people will be unable to use, and that's a fair question. First, it may be possible to use the technique without paying for a license. For example, if you're developing free software licensed under the GPL,[3] your software may be covered by IBM's statement of non-assertion.[4] Second, and most important, the explanation of the techniques shows some of the things that are important to think about when writing lock-free code, such as the costs of atomic operations.

[2] Maged M. Michael, U.S. Patent and Trademark Office application number 20040107227, "Method for efficient implementation of dynamic lock-free data structures with safe memory reclamation."

[3] GNU General Public License http://www.gnu.org/licenses/gpl.html.

[4] IBM Statement of Non-Assertion of Named Patents Against OSS, http://www.ibm.com/ibm/licensing/p http://www.ibm.com/ibm/licensing/patents/pledgedpatents.pdfatents/pledgedpatents.pdf.

So, are there any unpatented memory reclamation techniques that can be used with lock-free code? Luckily, there are. One such mechanism is reference counting.

7.2.4. Detecting nodes in use with reference counting

Back in section 7.2.2, you saw that the problem with deleting nodes is detecting which nodes are still being accessed by reader threads. If you could safely identify precisely which nodes were being referenced and when no threads were accessing these nodes, you could delete them. Hazard pointers tackle the problem by storing a list of the nodes in use. Reference